ArticlePopularityPrediction - ML Project

Vansh Chandwaney

2023-10-24

Case Objective and Data Description

The objective of this report is to predict the popularity of articles posted on Mashable, a renowned news website and entertainment company, by analyzing different predictive models. The focus is on automating the selection process of articles based on their potential to generate high shares, which directly impacts revenue.

The public dataset used in this analysis has details of 39645 articles posted on Mashable. Broadly, this dataset contained information about keywords used, data channels, the day of posting, hyperlinks, the language used, overall sentiment and the total shares for each article. There were 60 possible metrics in the dataset that could have been used to predict shares.

Loading necessary libraries

```r
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.3.1
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```r
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.2
```

```
## corrplot 0.92 loaded
```

```r
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.1
```

```
## Loading required package: lattice
```

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.1
```

```
## Warning: package 'tidyr' was built under R version 4.3.1
```

```
## Warning: package 'readr' was built under R version 4.3.1
```

```
## Warning: package 'purrr' was built under R version 4.3.1
```

```
## Warning: package 'forcats' was built under R version 4.3.1
```

```
## Warning: package 'lubridate' was built under R version 4.3.1
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ forcats   1.0.0     ✔ stringr   1.5.0
## ✔ lubridate 1.9.2     ✔ tibble    3.2.1
## ✔ purrr     1.0.2     ✔ tidyr     1.3.0
## ✔ readr     2.1.4
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ✖ purrr::lift()   masks caret::lift()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.3.1
```

```
## Loading required package: rpart
```

```r
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.3.1
```

```
##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##     slice
```

```r
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.1
```

```
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:ggplot2':
##
##     margin
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.3.1
```

```r
library(pROC)
```

```
## Warning: package 'pROC' was built under R version 4.3.1
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

Loading the dataset

```r
mashable_data <- read.csv("OnlineNewsPopularity.csv")
View(mashable_data)
```

Exploratory Data Analysis (EDA)

Summarising the data and checking for missing values

```r
summary(mashable_data)
```

```
##      url              timedelta     n_tokens_title n_tokens_content
##  Length:39644       Min.   :  8.0   Min.   : 2.0   Min.   :   0.0  
##  Class :character   1st Qu.:164.0   1st Qu.: 9.0   1st Qu.: 246.0  
##  Mode  :character   Median :339.0   Median :10.0   Median : 409.0  
##                     Mean   :354.5   Mean   :10.4   Mean   : 546.5  
##                     3rd Qu.:542.0   3rd Qu.:12.0   3rd Qu.: 716.0  
##                     Max.   :731.0   Max.   :23.0   Max.   :8474.0  
##  n_unique_tokens    n_non_stop_words    n_non_stop_unique_tokens
##  Min.   :  0.0000   Min.   :   0.0000   Min.   :  0.0000        
##  1st Qu.:  0.4709   1st Qu.:   1.0000   1st Qu.:  0.6257        
##  Median :  0.5392   Median :   1.0000   Median :  0.6905        
##  Mean   :  0.5482   Mean   :   0.9965   Mean   :  0.6892        
##  3rd Qu.:  0.6087   3rd Qu.:   1.0000   3rd Qu.:  0.7546        
##  Max.   :701.0000   Max.   :1042.0000   Max.   :650.0000        
##    num_hrefs       num_self_hrefs       num_imgs          num_videos   
##  Min.   :  0.00   Min.   :  0.000   Min.   :  0.000   Min.   : 0.00  
##  1st Qu.:  4.00   1st Qu.:  1.000   1st Qu.:  1.000   1st Qu.: 0.00  
##  Median :  8.00   Median :  3.000   Median :  1.000   Median : 0.00  
##  Mean   : 10.88   Mean   :  3.294   Mean   :  4.544   Mean   : 1.25  
##  3rd Qu.: 14.00   3rd Qu.:  4.000   3rd Qu.:  4.000   3rd Qu.: 1.00  
##  Max.   :304.00   Max.   :116.000   Max.   :128.000   Max.   :91.00  
##  average_token_length  num_keywords    data_channel_is_lifestyle
##  Min.   :0.000        Min.   : 1.000   Min.   :0.00000          
##  1st Qu.:4.478        1st Qu.: 6.000   1st Qu.:0.00000          
##  Median :4.664        Median : 7.000   Median :0.00000          
##  Mean   :4.548        Mean   : 7.224   Mean   :0.05295          
##  3rd Qu.:4.855        3rd Qu.: 9.000   3rd Qu.:0.00000          
##  Max.   :8.042        Max.   :10.000   Max.   :1.00000          
##  data_channel_is_entertainment data_channel_is_bus data_channel_is_socmed
##  Min.   :0.000                 Min.   :0.0000      Min.   :0.0000        
##  1st Qu.:0.000                 1st Qu.:0.0000      1st Qu.:0.0000        
##  Median :0.000                 Median :0.0000      Median :0.0000        
##  Mean   :0.178                 Mean   :0.1579      Mean   :0.0586        
##  3rd Qu.:0.000                 3rd Qu.:0.0000      3rd Qu.:0.0000        
##  Max.   :1.000                 Max.   :1.0000      Max.   :1.0000        
##  data_channel_is_tech data_channel_is_world   kw_min_min       kw_max_min    
##  Min.   :0.0000       Min.   :0.0000        Min.   : -1.00   Min.   :     0  
##  1st Qu.:0.0000       1st Qu.:0.0000        1st Qu.: -1.00   1st Qu.:   445  
##  Median :0.0000       Median :0.0000        Median : -1.00   Median :   660  
##  Mean   :0.1853       Mean   :0.2126        Mean   : 26.11   Mean   :  1154  
##  3rd Qu.:0.0000       3rd Qu.:0.0000        3rd Qu.:  4.00   3rd Qu.:  1000  
##  Max.   :1.0000       Max.   :1.0000        Max.   :377.00   Max.   :298400  
##    kw_avg_min        kw_min_max       kw_max_max       kw_avg_max    
##  Min.   :   -1.0   Min.   :     0   Min.   :     0   Min.   :     0  
##  1st Qu.:  141.8   1st Qu.:     0   1st Qu.:843300   1st Qu.:172847  
##  Median :  235.5   Median :  1400   Median :843300   Median :244572  
##  Mean   :  312.4   Mean   : 13612   Mean   :752324   Mean   :259282  
##  3rd Qu.:  357.0   3rd Qu.:  7900   3rd Qu.:843300   3rd Qu.:330980  
##  Max.   :42827.9   Max.   :843300   Max.   :843300   Max.   :843300  
##    kw_min_avg     kw_max_avg       kw_avg_avg    self_reference_min_shares
##  Min.   :  -1   Min.   :     0   Min.   :    0   Min.   :     0           
##  1st Qu.:   0   1st Qu.:  3562   1st Qu.: 2382   1st Qu.:   639           
##  Median :1024   Median :  4356   Median : 2870   Median :  1200           
##  Mean   :1117   Mean   :  5657   Mean   : 3136   Mean   :  3999           
##  3rd Qu.:2057   3rd Qu.:  6020   3rd Qu.: 3600   3rd Qu.:  2600           
##  Max.   :3613   Max.   :298400   Max.   :43568   Max.   :843300           
##  self_reference_max_shares self_reference_avg_sharess weekday_is_monday
##  Min.   :     0            Min.   :     0.0           Min.   :0.000    
##  1st Qu.:  1100            1st Qu.:   981.2           1st Qu.:0.000    
##  Median :  2800            Median :  2200.0           Median :0.000    
##  Mean   : 10329            Mean   :  6401.7           Mean   :0.168    
##  3rd Qu.:  8000            3rd Qu.:  5200.0           3rd Qu.:0.000    
##  Max.   :843300            Max.   :843300.0           Max.   :1.000    
##  weekday_is_tuesday weekday_is_wednesday weekday_is_thursday weekday_is_friday
##  Min.   :0.0000     Min.   :0.0000       Min.   :0.0000      Min.   :0.0000   
##  1st Qu.:0.0000     1st Qu.:0.0000       1st Qu.:0.0000      1st Qu.:0.0000   
##  Median :0.0000     Median :0.0000       Median :0.0000      Median :0.0000   
##  Mean   :0.1864     Mean   :0.1875       Mean   :0.1833      Mean   :0.1438   
##  3rd Qu.:0.0000     3rd Qu.:0.0000       3rd Qu.:0.0000      3rd Qu.:0.0000   
##  Max.   :1.0000     Max.   :1.0000       Max.   :1.0000      Max.   :1.0000   
##  weekday_is_saturday weekday_is_sunday   is_weekend         LDA_00       
##  Min.   :0.00000     Min.   :0.00000   Min.   :0.0000   Min.   :0.00000  
##  1st Qu.:0.00000     1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.02505  
##  Median :0.00000     Median :0.00000   Median :0.0000   Median :0.03339  
##  Mean   :0.06188     Mean   :0.06904   Mean   :0.1309   Mean   :0.18460  
##  3rd Qu.:0.00000     3rd Qu.:0.00000   3rd Qu.:0.0000   3rd Qu.:0.24096  
##  Max.   :1.00000     Max.   :1.00000   Max.   :1.0000   Max.   :0.92699  
##      LDA_01            LDA_02            LDA_03            LDA_04       
##  Min.   :0.00000   Min.   :0.00000   Min.   :0.00000   Min.   :0.00000  
##  1st Qu.:0.02501   1st Qu.:0.02857   1st Qu.:0.02857   1st Qu.:0.02857  
##  Median :0.03334   Median :0.04000   Median :0.04000   Median :0.04073  
##  Mean   :0.14126   Mean   :0.21632   Mean   :0.22377   Mean   :0.23403  
##  3rd Qu.:0.15083   3rd Qu.:0.33422   3rd Qu.:0.37576   3rd Qu.:0.39999  
##  Max.   :0.92595   Max.   :0.92000   Max.   :0.92653   Max.   :0.92719  
##  global_subjectivity global_sentiment_polarity global_rate_positive_words
##  Min.   :0.0000      Min.   :-0.39375          Min.   :0.00000           
##  1st Qu.:0.3962      1st Qu.: 0.05776          1st Qu.:0.02838           
##  Median :0.4535      Median : 0.11912          Median :0.03902           
##  Mean   :0.4434      Mean   : 0.11931          Mean   :0.03962           
##  3rd Qu.:0.5083      3rd Qu.: 0.17783          3rd Qu.:0.05028           
##  Max.   :1.0000      Max.   : 0.72784          Max.   :0.15549           
##  global_rate_negative_words rate_positive_words rate_negative_words
##  Min.   :0.000000           Min.   :0.0000      Min.   :0.0000     
##  1st Qu.:0.009615           1st Qu.:0.6000      1st Qu.:0.1852     
##  Median :0.015337           Median :0.7105      Median :0.2800     
##  Mean   :0.016612           Mean   :0.6822      Mean   :0.2879     
##  3rd Qu.:0.021739           3rd Qu.:0.8000      3rd Qu.:0.3846     
##  Max.   :0.184932           Max.   :1.0000      Max.   :1.0000     
##  avg_positive_polarity min_positive_polarity max_positive_polarity
##  Min.   :0.0000        Min.   :0.00000       Min.   :0.0000       
##  1st Qu.:0.3062        1st Qu.:0.05000       1st Qu.:0.6000       
##  Median :0.3588        Median :0.10000       Median :0.8000       
##  Mean   :0.3538        Mean   :0.09545       Mean   :0.7567       
##  3rd Qu.:0.4114        3rd Qu.:0.10000       3rd Qu.:1.0000       
##  Max.   :1.0000        Max.   :1.00000       Max.   :1.0000       
##  avg_negative_polarity min_negative_polarity max_negative_polarity
##  Min.   :-1.0000       Min.   :-1.0000       Min.   :-1.0000      
##  1st Qu.:-0.3284       1st Qu.:-0.7000       1st Qu.:-0.1250      
##  Median :-0.2533       Median :-0.5000       Median :-0.1000      
##  Mean   :-0.2595       Mean   :-0.5219       Mean   :-0.1075      
##  3rd Qu.:-0.1869       3rd Qu.:-0.3000       3rd Qu.:-0.0500      
##  Max.   : 0.0000       Max.   : 0.0000       Max.   : 0.0000      
##  title_subjectivity title_sentiment_polarity abs_title_subjectivity
##  Min.   :0.0000     Min.   :-1.00000         Min.   :0.0000        
##  1st Qu.:0.0000     1st Qu.: 0.00000         1st Qu.:0.1667        
##  Median :0.1500     Median : 0.00000         Median :0.5000        
##  Mean   :0.2824     Mean   : 0.07143         Mean   :0.3418        
##  3rd Qu.:0.5000     3rd Qu.: 0.15000         3rd Qu.:0.5000        
##  Max.   :1.0000     Max.   : 1.00000         Max.   :0.5000        
##  abs_title_sentiment_polarity     shares      
##  Min.   :0.0000               Min.   :     1  
##  1st Qu.:0.0000               1st Qu.:   946  
##  Median :0.0000               Median :  1400  
##  Mean   :0.1561               Mean   :  3395  
##  3rd Qu.:0.2500               3rd Qu.:  2800  
##  Max.   :1.0000               Max.   :843300  
```

```r
sum(is.na(mashable_data))
```

```
## [1] 0
```

Checking data types of each column

```r
column_data_types <- sapply(mashable_data, class)
column_data_types
```

```
##                           url                     timedelta 
##                   "character"                     "integer" 
##                n_tokens_title              n_tokens_content 
##                     "integer"                     "integer" 
##               n_unique_tokens              n_non_stop_words 
##                     "numeric"                     "numeric" 
##      n_non_stop_unique_tokens                     num_hrefs 
##                     "numeric"                     "integer" 
##                num_self_hrefs                      num_imgs 
##                     "integer"                     "integer" 
##                    num_videos          average_token_length 
##                     "integer"                     "numeric" 
##                  num_keywords     data_channel_is_lifestyle 
##                     "integer"                     "integer" 
## data_channel_is_entertainment           data_channel_is_bus 
##                     "integer"                     "integer" 
##        data_channel_is_socmed          data_channel_is_tech 
##                     "integer"                     "integer" 
##         data_channel_is_world                    kw_min_min 
##                     "integer"                     "integer" 
##                    kw_max_min                    kw_avg_min 
##                     "numeric"                     "numeric" 
##                    kw_min_max                    kw_max_max 
##                     "integer"                     "integer" 
##                    kw_avg_max                    kw_min_avg 
##                     "numeric"                     "numeric" 
##                    kw_max_avg                    kw_avg_avg 
##                     "numeric"                     "numeric" 
##     self_reference_min_shares     self_reference_max_shares 
##                     "numeric"                     "numeric" 
##    self_reference_avg_sharess             weekday_is_monday 
##                     "numeric"                     "integer" 
##            weekday_is_tuesday          weekday_is_wednesday 
##                     "integer"                     "integer" 
##           weekday_is_thursday             weekday_is_friday 
##                     "integer"                     "integer" 
##           weekday_is_saturday             weekday_is_sunday 
##                     "integer"                     "integer" 
##                    is_weekend                        LDA_00 
##                     "integer"                     "numeric" 
##                        LDA_01                        LDA_02 
##                     "numeric"                     "numeric" 
##                        LDA_03                        LDA_04 
##                     "numeric"                     "numeric" 
##           global_subjectivity     global_sentiment_polarity 
##                     "numeric"                     "numeric" 
##    global_rate_positive_words    global_rate_negative_words 
##                     "numeric"                     "numeric" 
##           rate_positive_words           rate_negative_words 
##                     "numeric"                     "numeric" 
##         avg_positive_polarity         min_positive_polarity 
##                     "numeric"                     "numeric" 
##         max_positive_polarity         avg_negative_polarity 
##                     "numeric"                     "numeric" 
##         min_negative_polarity         max_negative_polarity 
##                     "numeric"                     "numeric" 
##            title_subjectivity      title_sentiment_polarity 
##                     "numeric"                     "numeric" 
##        abs_title_subjectivity  abs_title_sentiment_polarity 
##                     "numeric"                     "numeric" 
##                        shares 
##                     "integer" 
```

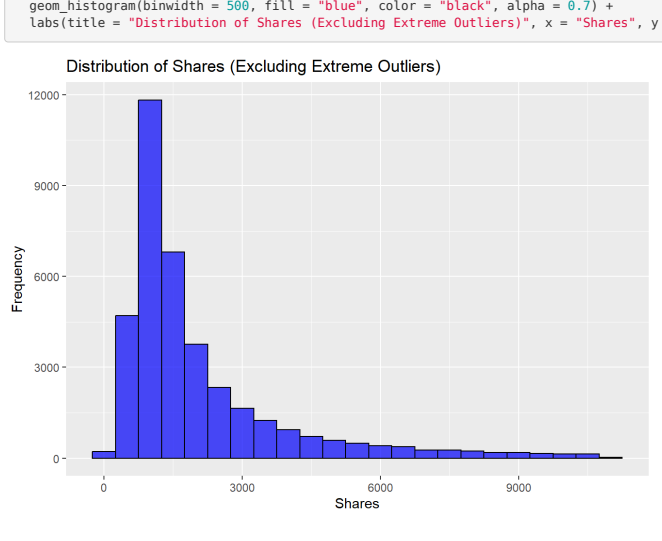
Creating a histogram of Article Shares

This would help visually visually illustrate the distribution of article shares, providing insights into the overall engagement levels of the content posted on Mashable.

```r
# Define the 95th percentile threshold
threshold <- quantile(mashable_data$shares, 0.95)

# Filter the dataset to exclude the top 5% extreme outliers
filtered_data <- mashable_data %>% filter(shares <= threshold)

# Histogram of shares excluding extreme outliers
ggplot(filtered_data, aes(x = shares)) +
  geom_histogram(binwidth = 500, fill = "blue", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Shares (Excluding Extreme Outliers)", x = "Shares", y = "Frequency")
```



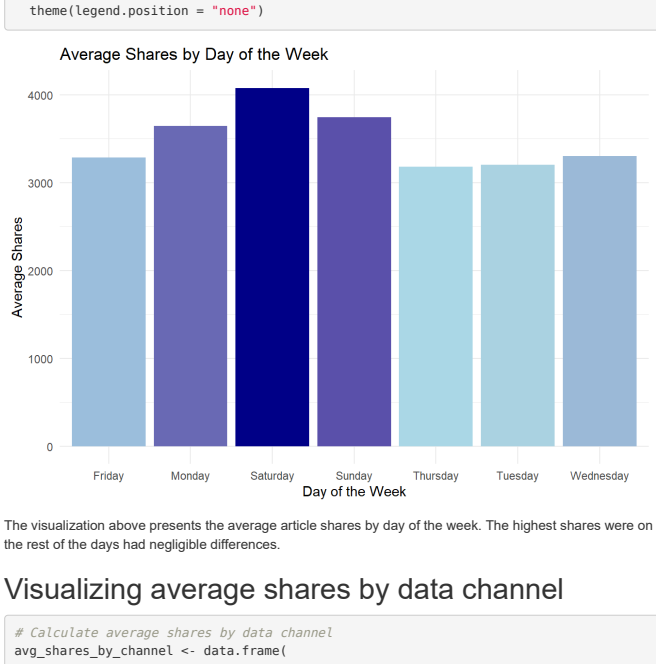
Changing data types of dummy variables to "factor"

```r
mashable_data$weekday_is_monday <- as.factor(mashable_data$weekday_is_monday)
mashable_data$weekday_is_tuesday <- as.factor(mashable_data$weekday_is_tuesday)
mashable_data$weekday_is_wednesday <- as.factor(mashable_data$weekday_is_wednesday)
mashable_data$weekday_is_thursday <- as.factor(mashable_data$weekday_is_thursday)
mashable_data$weekday_is_friday <- as.factor(mashable_data$weekday_is_friday)
mashable_data$weekday_is_saturday <- as.factor(mashable_data$weekday_is_saturday)
mashable_data$weekday_is_sunday <- as.factor(mashable_data$weekday_is_sunday)
mashable_data$is_weekend <- as.factor(mashable_data$is_weekend)
mashable_data$data_channel_is_bus <- as.factor(mashable_data$data_channel_is_bus)
mashable_data$data_channel_is_lifestyle <- as.factor(mashable_data$data_channel_is_lifestyle)
mashable_data$data_channel_is_entertainment <- as.factor(mashable_data$data_channel_is_entertainment)
mashable_data$data_channel_is_socmed <- as.factor(mashable_data$data_channel_is_socmed)
mashable_data$data_channel_is_tech <- as.factor(mashable_data$data_channel_is_tech)
mashable_data$data_channel_is_world <- as.factor(mashable_data$data_channel_is_world)
```

Visualizing average shares by day of the week

```r
# Calculate average shares by day of the week
avg_shares_by_day <- data.frame(
  weekday = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"),
  avg_shares = c(
    mean(mashable_data$shares[mashable_data$weekday_is_monday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_tuesday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_wednesday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_thursday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_friday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_saturday == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$weekday_is_sunday == 1], na.rm = TRUE)
  )
)

# Bar plot of average shares by day of the week with gradient colors based on size
ggplot(avg_shares_by_day, aes(x = weekday, y = avg_shares, fill = avg_shares)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(title = "Average Shares by Day of the Week", x = "Day of the Week", y = "Average Shares") +
  theme_minimal() +
  theme(legend.position = "none")
```

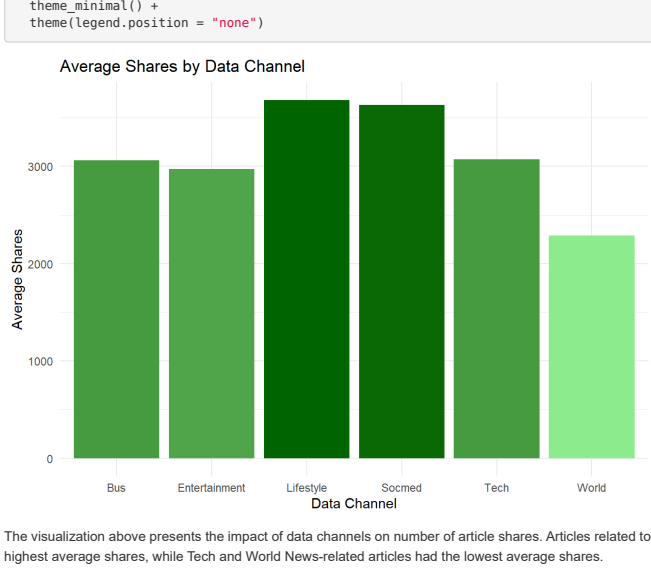


The visualization above presents the average article shares by day of the week. The highest shares were on Saturday, Sunday and Monday, and the rest of the days had negligible differences.

Visualizing average shares by data channel

```r
# Calculate average shares by data channel
avg_shares_by_channel <- data.frame(
  data_channel = c("Bus", "Lifestyle", "Entertainment", "Socmed", "Tech", "World"),
  avg_shares = c(
    mean(mashable_data$shares[mashable_data$data_channel_is_bus == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$data_channel_is_lifestyle == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$data_channel_is_entertainment == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$data_channel_is_socmed == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$data_channel_is_tech == 1], na.rm = TRUE),
    mean(mashable_data$shares[mashable_data$data_channel_is_world == 1], na.rm = TRUE)
  )
)

# Bar plot of average shares by data channel with gradient colors based on size
ggplot(avg_shares_by_channel, aes(x = data_channel, y = avg_shares, fill = avg_shares)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "lightgreen", high = "darkgreen") +
  labs(title = "Average Shares by Data Channel", x = "Data Channel", y = "Average Shares") +
  theme_minimal() +
  theme(legend.position = "none")
```



The visualization above presents the impact of data channels on number of article shares. Articles related to Lifestyle and Social Media had the highest average shares, while Tech and World News-related articles had the lowest average shares.

2. Data Pre-Processing

Removing columns "timedelta" and "url"

```r
# This is because they do not contribute to predicting shares
library(dplyr)
mashable_data <- mashable_data %>%
  select(-timedelta, -url)
```

Removing features from the dataset based on knowledge of the digital media industry

```r
mashable_data <- mashable_data %>%
  select(-average_token_length, -kw_min_min, -kw_max_min, -kw_min_max, -kw_max_max, -min_positive_polarity, -max_
positive_polarity, -min_negative_polarity, -max_negative_polarity, -is_weekend, -self_reference_min_shares, -self
_reference_max_shares)
```

### Splitting data into test and train sets

```r
dataset_split <- sample(c(TRUE, FALSE), nrow(mashable_data), replace=TRUE, prob=c(0.75,0.25))
train <- mashable_data[dataset_split, ]
test <- mashable_data[!dataset_split, ]
```

Developing Predictive Models

Performing linear regression

```r
linear_reg_model <- lm(shares ~ ., data = train)
summary(linear_reg_model)
```

```
##
## Call:
## lm(formula = shares ~ ., data = train)
##
## Residuals:
##    Min     1Q Median     3Q    Max 
## -37937  -2302  -1241    -92 684256 
##
## Coefficients: (1 not defined because of singularities)
##                                 Estimate Std. Error t value Pr(>|t|)    
## (Intercept)                    3.181e+06  7.050e+06   0.451 0.651857    
## n_tokens_title                 9.857e+01  3.326e+01   2.964 0.003041 ** 
## n_tokens_content               7.997e-01  2.517e-01   3.178 0.001487 ** 
## n_unique_tokens                4.214e+03  2.008e+03   2.099 0.035839 *  
## n_non_stop_words              -4.692e+03  6.813e+03  -0.689 0.491069    
## n_non_stop_unique_tokens      -1.914e+03  1.766e+03  -1.084 0.278514    
## num_hrefs                      1.960e+01  7.513e+00   2.609 0.009078 ** 
## num_self_hrefs                -5.420e+01  2.022e+01  -2.681 0.007343 ** 
## num_imgs                       3.149e+00  1.041e+01   0.303 0.762221    
## num_videos                     6.278e+00  1.820e+01   0.345 0.730193    
## num_keywords                   6.742e+01  4.170e+01   1.617 0.105923    
## data_channel_is_lifestyle1    -1.248e+03  4.600e+02  -2.714 0.006660 ** 
## data_channel_is_entertainment1 -1.253e+03  2.894e+02  -4.330 1.50e-05 ***
## data_channel_is_bus1          -8.341e+02  4.471e+02  -1.866 0.062089 .  
## data_channel_is_socmed1       -8.264e+02  4.310e+02  -1.917 0.055196 .  
## data_channel_is_tech1         -5.612e+02  4.315e+02  -1.301 0.193421    
## data_channel_is_world1        -6.215e+02  4.336e+02  -1.433 0.151750    
## kw_avg_min                     1.931e-01  1.332e-01   1.449 0.147319    
## kw_avg_max                    -2.867e-03  7.169e-04  -3.999 6.38e-05 ***
## kw_min_avg                    -3.095e-01  8.470e-02  -3.654 0.000258 ***
## kw_max_avg                    -1.967e-01  2.704e-02  -7.275 3.56e-13 ***
## kw_avg_avg                     1.646e+00  1.572e-01  10.469  < 2e-16 ***
## self_reference_avg_sharess     2.400e-02  2.799e-03   8.574  < 2e-16 ***
## weekday_is_monday1             4.020e+02  3.094e+02   1.299 0.193810    
## weekday_is_tuesday1           -1.786e+02  3.047e+02  -0.586 0.557875    
## weekday_is_wednesday1         -1.240e+02  3.046e+02  -0.407 0.683913    
## weekday_is_thursday1          -1.256e+02  3.057e+02  -0.411 0.681215    
## weekday_is_friday1            -1.812e+02  3.159e+02  -0.574 0.566198    
## weekday_is_saturday1           4.813e+02  3.762e+02   1.279 0.200747    
## weekday_is_sunday1                    NA         NA      NA       NA    
## LDA_00                        -3.181e+06  7.050e+06  -0.451 0.651878    
## LDA_01                        -3.182e+06  7.050e+06  -0.451 0.651761    
## LDA_02                        -3.182e+06  7.050e+06  -0.451 0.651703    
## LDA_03                        -3.181e+06  7.050e+06  -0.451 0.651831    
## LDA_04                        -3.181e+06  7.050e+06  -0.451 0.651808    
## global_subjectivity            2.190e+03  9.848e+02   2.224 0.026160 *  
## global_sentiment_polarity      7.328e+02  1.900e+03   0.386 0.699762    
## global_rate_positive_words    -1.611e+04  8.261e+03  -1.950 0.051207 .  
## global_rate_negative_words     9.777e+03  1.571e+04   0.622 0.533674    
## rate_positive_words            2.543e+03  6.770e+03   0.376 0.707190    
## rate_negative_words            1.814e+03  6.822e+03   0.266 0.790321    
## avg_positive_polarity         -2.095e+03  1.253e+03  -1.672 0.094530 .  
## avg_negative_polarity         -2.735e+03  7.288e+02  -3.753 0.000175 ***
## title_subjectivity             3.101e+00  3.223e+02   0.010 0.992324    
## title_sentiment_polarity       3.731e+02  2.945e+02   1.267 0.205176    
## abs_title_subjectivity         7.093e+02  4.269e+02   1.661 0.096633 .  
## abs_title_sentiment_polarity   4.754e+02  4.652e+02   1.022 0.306892    
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11670 on 29760 degrees of freedom
## Multiple R-squared:  0.02414,    Adjusted R-squared:  0.02267 
## F-statistic: 16.36 on 45 and 29760 DF,  p-value: < 2.2e-16
```

Since the r^2 of the linear regression model was low, a feature selection method will be used to run a new linear regression model with less variables that are more relevant.

```r
# Subsetting mashable_data without binary variables related to day of week and data channel for the purpose of fe
ature selection. These will be added back later for the second linear regression model as these attributes do aff
ect shares of an article, as is seen in the earlier in the visualisations.

data_for_feature_sel <- mashable_data[, c(1:10, 17:22, 30:46)]
```

Using XGBoost to establish feature importance, with nrounds as 100 and 200

```r
# Importance scores will be assessed with nrounds set to 100 and 200
library(caret)
library(tidyverse)
library(rpart.plot)
library(xgboost)

xgb_model_100 <- xgboost(data = data.matrix(data_for_feature_sel),
                 label = mashable_data[, 47],
                 nrounds = 100,
                 objective = "reg:squarederror")
```

```
## [1]  train-rmse:10938.496738 
## [2]  train-rmse:10347.215277 
## [3]  train-rmse:9980.484504 
## [4]  train-rmse:9489.638230 
## [5]  train-rmse:9244.701368 
## [6]  train-rmse:8961.355598 
## [7]  train-rmse:8730.517940 
## [8]  train-rmse:8550.264985 
## [9]  train-rmse:8342.440975 
## [10] train-rmse:8310.880609 
## [11] train-rmse:8232.229102 
## [12] train-rmse:8104.627210 
## [13] train-rmse:8085.987650 
## [14] train-rmse:7999.086601 
## [15] train-rmse:7769.318814 
## [16] train-rmse:7744.270068 
## [17] train-rmse:7675.981202 
## [18] train-rmse:7569.661347 
## [19] train-rmse:7514.715810 
## [20] train-rmse:7446.879476 
## [21] train-rmse:7335.353281 
## [22] train-rmse:7306.407919 
## [23] train-rmse:7256.751450 
## [24] train-rmse:7150.466385 
## [25] train-rmse:7125.950057 
## [26] train-rmse:7099.194220 
## [27] train-rmse:7072.902916 
## [28] train-rmse:7043.053439 
## [29] train-rmse:6983.072069 
## [30] train-rmse:6922.069366 
## [31] train-rmse:6910.539933 
## [32] train-rmse:6865.777510 
## [33] train-rmse:6852.820994 
## [34] train-rmse:6834.636736 
## [35] train-rmse:6802.164831 
## [36] train-rmse:6785.139918 
## [37] train-rmse:6738.162464 
## [38] train-rmse:6699.457073 
## [39] train-rmse:6652.387997 
## [40] train-rmse:6600.353839 
## [41] train-rmse:6507.127172 
## [42] train-rmse:6480.090606 
## [43] train-rmse:6473.699331 
## [44] train-rmse:6460.699662 
## [45] train-rmse:6390.242013 
## [46] train-rmse:6358.108607 
## [47] train-rmse:6351.324822 
## [48] train-rmse:6343.065405 
## [49] train-rmse:6308.140261 
## [50] train-rmse:6244.572848 
## [51] train-rmse:6230.080245 
## [52] train-rmse:6195.561560 
## [53] train-rmse:6116.692624 
## [54] train-rmse:6093.047716 
## [55] train-rmse:6040.917964 
## [56] train-rmse:5983.850131 
## [57] train-rmse:5959.935462 
## [58] train-rmse:5944.695214 
## [59] train-rmse:5920.714483 
## [60] train-rmse:5905.012337 
## [61] train-rmse:5882.490108 
## [62] train-rmse:5878.877775 
## [63] train-rmse:5829.320176 
## [64] train-rmse:5790.444760 
## [65] train-rmse:5784.233101 
## [66] train-rmse:5746.985831 
## [67] train-rmse:5737.247420 
## [68] train-rmse:5706.221765 
## [69] train-rmse:5681.997659 
## [70] train-rmse:5668.052134 
## [71] train-rmse:5645.004317 
## [72] train-rmse:5616.398698 
## [73] train-rmse:5475.880992 
## [74] train-rmse:5446.606274 
## [75] train-rmse:5415.452751 
## [76] train-rmse:5398.704119 
## [77] train-rmse:5323.118164 
## [78] train-rmse:5260.235827 
## [79] train-rmse:5195.904616 
## [80] train-rmse:5184.565920 
## [81] train-rmse:5157.583535 
## [82] train-rmse:5148.905704 
## [83] train-rmse:5106.845842 
## [84] train-rmse:5078.864728 
## [85] train-rmse:5072.208563 
## [86] train-rmse:5032.129517 
## [87] train-rmse:5026.675012 
## [88] train-rmse:5014.542520 
## [89] train-rmse:4958.813091 
## [90] train-rmse:4951.852624 
## [91] train-rmse:4942.923924 
## [92] train-rmse:4939.039249 
## [93] train-rmse:4917.603481 
## [94] train-rmse:4903.516555 
## [95] train-rmse:4898.947328 
## [96] train-rmse:4890.349673 
## [97] train-rmse:4875.861307 
## [98] train-rmse:4859.236055 
## [99] train-rmse:4832.223636 
## [100]    train-rmse:4822.832784 
```

```r
xgb_model_200 <- xgboost(data = data.matrix(data_for_feature_sel),
                 label = mashable_data[, 47],
                 nrounds = 200,
                 objective = "reg:squarederror")
```

```
## [1]  train-rmse:10938.496738 
## [2]  train-rmse:10347.215277 
## [3]  train-rmse:9980.484504 
## [4]  train-rmse:9489.638230 
## [5]  train-rmse:9244.701368 
## [6]  train-rmse:8961.355598 
## [7]  train-rmse:8730.517940 
## [8]  train-rmse:8550.264985 
## [9]  train-rmse:8342.440975 
## [10] train-rmse:8310.880609 
## [11] train-rmse:8232.229102 
## [12] train-rmse:8104.627210 
## [13] train-rmse:8085.987650 
## [14] train-rmse:7999.086601 
## [15] train-rmse:7769.318814 
## [16] train-rmse:7744.270068 
## [17] train-rmse:7675.981202 
## [18] train-rmse:7569.661347 
## [19] train-rmse:7514.715810 
## [20] train-rmse:7446.879476 
## [21] train-rmse:7335.353281 
## [22] train-rmse:7306.407919 
## [23] train-rmse:7256.751450 
## [24] train-rmse:7150.466385 
## [25] train-rmse:7125.950057 
## [26] train-rmse:7099.194220 
## [27] train-rmse:7072.902916 
## [28] train-rmse:7043.053439 
## [29] train-rmse:6983.072069 
## [30] train-rmse:6922.069366 
## [31] train-rmse:6910.539933 
## [32] train-rmse:6865.777510 
## [33] train-rmse:6852.820994 
## [34] train-rmse:6834.636736 
## [35] train-rmse:6802.164831 
## [36] train-rmse:6785.139918 
## [37] train-rmse:6738.162464 
## [38] train-rmse:6699.457073 
## [39] train-rmse:6652.387997 
## [40] train-rmse:6600.353839 
## [41] train-rmse:6507.127172 
## [42] train-rmse:6480.090606 
## [43] train-rmse:6473.699331 
## [44] train-rmse:6460.699662 
## [45] train-rmse:6390.242013 
## [46] train-rmse:6358.108607 
## [47] train-rmse:6351.324822 
## [48] train-rmse:6343.065405 
## [49] train-rmse:6308.140261 
## [50] train-rmse:6244.572848 
## [51] train-rmse:6230.080245 
## [52] train-rmse:6195.561560 
## [53] train-rmse:6116.692624 
## [54] train-rmse:6093.047716 
## [55] train-rmse:6040.917964 
## [56] train-rmse:5983.850131 
## [57] train-rmse:5959.935462 
## [58] train-rmse:5944.695214 
## [59] train-rmse:5920.714483 
## [60] train-rmse:5905.012337 
## [61] train-rmse:5882.490108 
## [62] train-rmse:5878.877775 
## [63] train-rmse:5829.320176 
## [64] train-rmse:5790.444760 
## [65] train-rmse:5784.233101 
## [66] train-rmse:5746.985831 
## [67] train-rmse:5737.247420 
## [68] train-rmse:5706.221765 
## [69] train-rmse:5681.997659 
## [70] train-rmse:5668.052134 
## [71] train-rmse:5645.004317 
## [72] train-rmse:5616.398698 
## [73] train-rmse:5475.880992 
## [74] train-rmse:5446.606274 
## [75] train-rmse:5415.452751 
## [76] train-rmse:5398.704119 
## [77] train-rmse:5323.118164 
## [78] train-rmse:5260.235827 
## [79] train-rmse:5195.904616 
## [80] train-rmse:5184.565920 
## [81] train-rmse:5157.583535 
## [82] train-rmse:5148.905704 
## [83] train-rmse:5106.845842 
## [84] train-rmse:5078.864728 
## [85] train-rmse:5072.208563 
## [86] train-rmse:5032.129517 
## [87] train-rmse:5026.675012 
## [88] train-rmse:5014.542520 
## [89] train-rmse:4958.813091 
## [90] train-rmse:4951.852624 
## [91] train-rmse:4942.923924 
## [92] train-rmse:4939.039249 
## [93] train-rmse:4917.603481 
## [94] train-rmse:4903.516555 
## [95] train-rmse:4898.947328 
## [96] train-rmse:4890.349673 
## [97] train-rmse:4875.861307 
## [98] train-rmse:4859.236055 
## [99] train-rmse:4832.223636 
## [100]    train-rmse:4822.832784 
## [101]    train-rmse:4817.602073 
## [102]    train-rmse:4808.884807 
## [103]    train-rmse:4787.090639 
## [104]    train-rmse:4768.304483 
## [105]    train-rmse:4729.656624 
## [106]    train-rmse:4656.629379 
## [107]    train-rmse:4633.405280 
## [108]    train-rmse:4618.363068 
## [109]    train-rmse:4612.989902 
## [110]    train-rmse:4590.886407 
## [111]    train-rmse:4564.795708 
## [112]    train-rmse:4543.584615 
## [113]    train-rmse:4520.321380 
## [114]    train-rmse:4498.815410 
## [115]    train-rmse:4491.484163 
## [116]    train-rmse:4481.839363 
## [117]    train-rmse:4463.625683 
## [118]    train-rmse:4445.313577 
## [119]    train-rmse:4438.841117 
## [120]    train-rmse:4431.537751 
## [121]    train-rmse:4417.563427 
## [122]    train-rmse:4401.893479 
## [123]    train-rmse:4380.735848 
## [124]    train-rmse:4337.780434 
## [125]    train-rmse:4310.192798 
## [126]    train-rmse:4305.382929 
## [127]    train-rmse:4302.013908 
## [128]    train-rmse:4282.434808 
## [129]    train-rmse:4278.854609 
## [130]    train-rmse:4268.690647 
## [131]    train-rmse:4256.690318 
## [132]    train-rmse:4233.317451 
## [133]    train-rmse:4232.285527 
## [134]    train-rmse:4221.837171 
## [135]    train-rmse:4210.700836 
## [136]    train-rmse:4200.913405 
## [137]    train-rmse:4185.329409 
## [138]    train-rmse:4180.659420 
## [139]    train-rmse:4140.526706 
## [140]    train-rmse:4111.123661 
## [141]    train-rmse:4083.108604 
## [142]    train-rmse:4066.833075 
## [143]    train-rmse:4059.102410 
## [144]    train-rmse:4042.731552 
## [145]    train-rmse:4008.567501 
## [146]    train-rmse:3993.919927 
## [147]    train-rmse:3984.283924 
## [148]    train-rmse:3959.261213 
## [149]    train-rmse:3943.741029 
## [150]    train-rmse:3922.660294 
## [151]    train-rmse:3909.270302 
## [152]    train-rmse:3899.589088 
## [153]    train-rmse:3886.466946 
## [154]    train-rmse:3881.540367 
## [155]    train-rmse:3874.245109 
## [156]    train-rmse:3856.045721 
## [157]    train-rmse:3826.807137 
## [158]    train-rmse:3811.215374 
## [159]    train-rmse:3809.326575 
## [160]    train-rmse:3798.636395 
## [161]    train-rmse:3790.854313 
## [162]    train-rmse:3774.220656 
## [163]    train-rmse:3766.309807 
## [164]    train-rmse:3765.920975 
## [165]    train-rmse:3759.480373 
## [166]    train-rmse:3745.514719 
## [167]    train-rmse:3733.538276 
## [168]    train-rmse:3728.610786 
## [169]    train-rmse:3720.550213 
## [170]    train-rmse:3711.270769 
## [171]    train-rmse:3699.650114 
## [172]    train-rmse:3685.547683 
## [173]    train-rmse:3670.588635 
## [174]    train-rmse:3663.124960 
## [175]    train-rmse:3652.285477 
## [176]    train-rmse:3646.541551 
## [177]    train-rmse:3635.205372 
## [178]    train-rmse:3624.911906 
## [179]    train-rmse:3612.327878 
## [180]    train-rmse:3598.896285 
## [181]    train-rmse:3580.551372 
## [182]    train-rmse:3561.146229 
## [183]    train-rmse:3541.881674 
## [184]    train-rmse:3528.648989 
## [185]    train-rmse:3512.328408 
## [186]    train-rmse:3507.305639 
## [187]    train-rmse:3490.479438 
## [188]    train-rmse:3484.392486 
## [189]    train-rmse:3465.269406 
## [190]    train-rmse:3455.140335 
## [191]    train-rmse:3453.223942 
## [192]    train-rmse:3450.499813 
## [193]    train-rmse:3429.657456 
## [194]    train-rmse:3415.769356 
## [195]    train-rmse:3408.133428 
## [196]    train-rmse:3404.908337 
## [197]    train-rmse:3393.823350 
## [198]    train-rmse:3385.782706 
## [199]    train-rmse:3383.422963 
## [200]    train-rmse:3379.279058 
```

Checking importance scores

```r
importance_scores_100 <- xgb.importance(model = xgb_model_100) # https://www.projectpro.io/recipes/visualise-xgbo
ost-feature-importance-r was used as a reference for this code.
head(importance_scores_100, 10)
```

```
##                        Feature       Gain      Cover  Frequency
##  1: self_reference_avg_sharess 0.10358278 0.08008498 0.07264640
##  2:            n_unique_tokens 0.08505804 0.02679014 0.04274771
##  3:           n_tokens_content 0.07930283 0.03735290 0.06745738
##  4:                 kw_max_avg 0.07892939 0.06397192 0.04843094
##  5:             n_tokens_title 0.06725192 0.01052551 0.08524833
##  6:   n_non_stop_unique_tokens 0.06676711 0.02110248 0.03311095
##  7:                 kw_avg_max 0.05756963 0.05411715 0.05114900
##  8:                 kw_avg_avg 0.05558088 0.06359255 0.04348900
##  9:                 kw_avg_min 0.05320747 0.03858868 0.02940450
## 10:                     LDA_03 0.04677263 0.07294227 0.04348900
```

```r
cat("Total gain of top 10 variables using XGBoost with 100 boosting rounds is", sum(importance_scores_100$Gain[1:
10]), "\n")
```

```
## Total gain of top 10 variables using XGBoost with 100 boosting rounds is 0.6940227
```

```r
importance_scores_200 <- xgb.importance(model = xgb_model_200) # https://www.projectpro.io/recipes/visualise-xgbo
ost-feature-importance-r was used as a reference for this code.
head(importance_scores_200, 10)
```

```
##                        Feature       Gain       Cover  Frequency
##  1: self_reference_avg_sharess 0.09836154 0.068358628 0.05989521
##  2:            n_unique_tokens 0.07999186 0.038778325 0.04120029
##  3:           n_tokens_content 0.07653432 0.037828808 0.06775423
##  4:                 kw_max_avg 0.07629753 0.053060272 0.04858300
##  5:             n_tokens_title 0.06681829 0.006477875 0.08144796
##  6:   n_non_stop_unique_tokens 0.06460921 0.031776110 0.03691355
##  7:                 kw_avg_max 0.05692848 0.054346693 0.04822577
##  8:                 kw_avg_avg 0.05448581 0.071206150 0.04358181
##  9:                 kw_avg_min 0.05280072 0.038202670 0.03524649
## 10:                     LDA_03 0.04807424 0.072425700 0.04512979
```

```r
cat("Total gain of top 10 variables using XGBoost with 200 boosting rounds is",sum(importance_scores_200$Gain[1:1
0]))
```

```
## Total gain of top 10 variables using XGBoost with 200 boosting rounds is 0.674902
```

The total gain for the top 10 variables as the boosting rounds increases from 100 to 200, decreases from 0.694 to 0.674. This may indicate overfitting due to the higher complexity of the model. Hence, the model with 100 boosting rounds will be used.

Selecting variables that have a gain of more than 0.05 (the variables that have a contribution to the model that is greater than 5%)

```r
xgb_selected_variables<- importance_scores_100[importance_scores_100$Gain>0.05, 1]
```

Creating new testing and training datasets

```r
# keeping only relevant variables in the training and testing dataset. The dummy variables for day of the week an
d data channels have been added back.
training_data_new<-train[,c(11:16, 23:29, 22, 3, 20, 2, 1, 5, 17, 18, 21, 47)]
testing_data_new<-test[,c(11:16, 23:29, 22, 3, 20, 2, 1, 5, 17, 18, 21, 47)]
```

Conducting multiple linear regression

```r
linear_reg_model_2 <- lm(shares ~ ., data = training_data_new)
summary(linear_reg_model_2)
```

```
##
## Call:
## lm(formula = shares ~ ., data = training_data_new)
##
## Residuals:
##    Min     1Q Median     3Q    Max 
## -36281  -2224  -1310   -240 684042 
##
## Coefficients: (1 not defined because of singularities)
##                                 Estimate Std. Error t value Pr(>|t|)    
## (Intercept)                    9.466e+02  5.991e+02   1.580 0.114114    
## data_channel_is_lifestyle1    -1.576e+03  3.649e+02  -4.319 1.58e-05 ***
## data_channel_is_entertainment1 -1.845e+03  2.619e+02  -7.044 1.91e-12 ***
## data_channel_is_bus1          -1.160e+03  2.814e+02  -4.123 3.75e-05 ***
## data_channel_is_socmed1       -1.442e+03  3.490e+02  -4.133 3.60e-05 ***
## data_channel_is_tech1         -1.355e+03  2.724e+02  -4.972 6.67e-07 ***
## data_channel_is_world1        -1.771e+03  2.783e+02  -6.365 1.98e-10 ***
## weekday_is_monday1             4.092e+02  3.090e+02   1.324 0.185437    
## weekday_is_tuesday1           -1.609e+02  3.042e+02  -0.529 0.596991    
## weekday_is_wednesday1         -1.229e+02  3.041e+02  -0.404 0.686074    
## weekday_is_thursday1          -1.061e+02  3.052e+02  -0.348 0.728125    
## weekday_is_friday1            -1.391e+02  3.156e+02  -0.441 0.659302    
## weekday_is_saturday1           4.853e+02  3.763e+02   1.290 0.197183    
## weekday_is_sunday1                    NA         NA      NA       NA    
## self_reference_avg_sharess     2.456e-02  2.795e-03   8.788  < 2e-16 ***
## n_unique_tokens                2.417e+03  1.255e+03   1.926 0.054071 .  
## kw_max_avg                    -1.555e-01  2.318e-02  -6.707 2.02e-11 ***
## n_tokens_content               5.594e-01  1.515e-01   3.692 0.000223 ***
## n_tokens_title                 9.420e+01  3.271e+01   2.880 0.003981 ** 
## n_non_stop_unique_tokens      -2.602e+03  1.353e+03  -1.923 0.054475 .  
## kw_avg_min                     1.551e-01  1.327e-01   1.169 0.242489    
## kw_avg_max                    -3.379e-03  6.638e-04  -5.090 3.60e-07 ***
## kw_avg_avg                     1.442e+00  1.226e-01  11.770  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11690 on 29784 degrees of freedom
## Multiple R-squared:  0.02088,    Adjusted R-squared:  0.02019 
## F-statistic: 30.25 on 21 and 29784 DF,  p-value: < 2.2e-16
```

Due to a poor r^2 even after feature selection, the linear regression model will not be used for analysis.

Logistic regression: classifying popularity

As direct predictions for popularity were inaccurate (possibly due to features outside the dataset that would affect shares), classification models were used for analysis. The models discussed hereafter will thus attempt to predict whether or not an article would have a high number of shares, rather than predicting the number of shares generated. To go ahead with this analysis, all articles that were shared more than 1000 times were classified as 'Popular' and all articles that had <1000 shares were deemed 'Unpopular'.

```r
train$is_popular<- ifelse(train$shares>1000, 1, 0)
test$is_popular<- ifelse(test$shares>1000, 1, 0)
train$is_popular<-as.factor(train$is_popular)
test$is_popular<-as.factor(test$is_popular)
```

Preparing data for classification models

```r
data_for_logistic_reg<- mashable_data
data_for_logistic_reg$is_popular<- ifelse(data_for_logistic_reg$shares>1000, 1, 0)
data_for_logistic_reg$is_popular<-as.factor(data_for_logistic_reg$is_popular)
data_for_logistic_reg <- data_for_logistic_reg %>%
  select(-shares)

# Splitting data into test and train
set.seed(50)
test_train_split_log <- sample(c(TRUE, FALSE), nrow(data_for_logistic_reg), replace=TRUE, prob=c(0.75,0.25))
log_training_data <- data_for_logistic_reg[test_train_split_log, ]
log_testing_data <- data_for_logistic_reg[!test_train_split_log, ]
```

Performing logistic regression with 46 independent variables

```r
logistic_reg_model <- glm(is_popular ~ ., data = log_training_data, family = binomial)
summary(logistic_reg_model)
```

```
##
## Call:
## glm(formula = is_popular ~ ., family = binomial, data = log_training_data)
##
## Coefficients: (1 not defined because of singularities)
##                                 Estimate Std. Error z value Pr(>|z|)    
## (Intercept)                    2.576e+02  1.310e+03   0.197 0.844115    
## n_tokens_title                 4.562e-03  6.558e-03   0.696 0.486642    
## n_tokens_content               8.902e-05  5.399e-05   1.649 0.099195 .  
## n_unique_tokens               -3.153e-01  4.053e-01  -0.778 0.436579    
## n_non_stop_words               3.498e-01  1.266e+00   0.276 0.782206    
## n_non_stop_unique_tokens      -6.023e-01  3.555e-01  -1.694 0.090187 .  
## num_hrefs                      8.471e-03  1.686e-03   5.023 5.09e-07 ***
## num_self_hrefs                -1.744e-02  4.415e-03  -3.951 7.78e-05 ***
## num_imgs                       1.149e-03  2.170e-03   0.529 0.596614    
## num_videos                     6.633e-03  3.746e-03   1.771 0.076575 .  
## num_keywords                   1.215e-02  8.296e-03   1.465 0.143035    
## data_channel_is_lifestyle1     8.084e-02  9.766e-02   0.828 0.407799    
## data_channel_is_entertainment1 -2.971e-01  5.624e-02  -5.283 1.27e-07 ***
## data_channel_is_bus1          -6.893e-02  9.161e-02  -0.752 0.451810    
## data_channel_is_socmed1        1.226e+00  1.028e-01  11.917  < 2e-16 ***
## data_channel_is_tech1          5.481e-01  8.850e-02   6.193 5.90e-10 ***
## data_channel_is_world1         2.014e-02  8.602e-02   0.234 0.814894    
## kw_avg_min                     3.434e-05  3.760e-05   0.914 0.360974    
## kw_avg_max                    -1.490e-06  1.475e-07 -10.101  < 2e-16 ***
## kw_min_avg                    -6.244e-05  1.803e-05  -3.463 0.000534 ***
## kw_max_avg                    -9.451e-05  6.094e-06 -15.509  < 2e-16 ***
## kw_avg_avg                     7.222e-04  3.570e-05  20.229  < 2e-16 ***
## self_reference_avg_sharess     6.689e-06  1.146e-06   5.839 5.26e-09 ***
## weekday_is_monday1            -1.060e+00  7.115e-02 -14.902  < 2e-16 ***
## weekday_is_tuesday1           -1.188e+00  7.052e-02 -16.847  < 2e-16 ***
## weekday_is_wednesday1         -1.221e+00  7.041e-02 -17.348  < 2e-16 ***
## weekday_is_thursday1          -1.185e+00  7.051e-02 -16.798  < 2e-16 ***
## weekday_is_friday1            -8.586e-01  7.298e-02 -11.764  < 2e-16 ***
## weekday_is_saturday1           1.892e-01  9.735e-02   1.943 0.051969 .  
## weekday_is_sunday1                    NA         NA      NA       NA    
## LDA_00                        -2.567e+02  1.310e+03  -0.196 0.844669    
## LDA_01                        -2.577e+02  1.310e+03  -0.197 0.844101    
## LDA_02                        -2.578e+02  1.310e+03  -0.197 0.844042    
## LDA_03                        -2.577e+02  1.310e+03  -0.197 0.844096    
## LDA_04                        -2.570e+02  1.310e+03  -0.196 0.844467    
## global_subjectivity            8.739e-01  1.929e-01   4.531 5.88e-06 ***
## global_sentiment_polarity      1.163e-01  3.796e-01   0.306 0.759306    
## global_rate_positive_words    -8.887e-01  1.666e+00  -0.533 0.593854    
## global_rate_negative_words     3.081e+00  3.126e+00   0.986 0.324367    
## rate_positive_words           -2.006e-01  1.255e+00  -0.160 0.873054    
## rate_negative_words           -5.754e-01  1.265e+00  -0.455 0.649300    
## avg_positive_polarity         -3.437e-01  2.419e-01  -1.421 0.155371    
## avg_negative_polarity         -1.415e-01  1.439e-01  -0.984 0.325191    
## title_subjectivity             8.989e-02  6.434e-02   1.397 0.162410    
## title_sentiment_polarity       1.460e-01  5.805e-02   2.516 0.011867 *  
## abs_title_subjectivity         2.788e-01  8.488e-02   3.284 0.001022 ** 
## abs_title_sentiment_polarity   1.688e-02  9.248e-02   0.182 0.855196    
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 37131  on 29788  degrees of freedom
## Residual deviance: 33410  on 29743  degrees of freedom
## AIC: 33502
##
## Number of Fisher Scoring iterations: 8
```

Assessing the model's predictive ability

```r
library(caret)

predicted_prob <- predict(logistic_reg_model, newdata = log_training_data, type = "response") # this code was tak
en from ChatGPT
accuracy <- mean(predicted_prob >= 0.5)  # Assuming a threshold of 0.5 for binary classification. this code was t
aken from ChatGPT
cat("The accuracy rate of this model is", accuracy*100, "%")
```

```
## The accuracy rate of this model is 84.66212 %
```

Performing logistic regression model with 9 independent variables selected using XGBoost

```r
# Preparing data
data_for_logistic_reg2<- data_for_logistic_reg
data_for_logistic_reg2 <- data_for_logistic_reg2 %>%
  select(self_reference_avg_sharess, n_unique_tokens, n_tokens_content, kw_max_avg, n_tokens_title, n_non_stop_un
ique_tokens, kw_avg_min, kw_avg_max, kw_avg_avg, is_popular)
```

Creating new testing and training datasets

```r
set.seed(52)
test_train_split_log_2 <- sample(c(TRUE, FALSE), nrow(data_for_logistic_reg2), replace=TRUE, prob=c(0.75,0.25))
log_training_data_2 <- data_for_logistic_reg2[test_train_split_log_2, ]
log_testing_data_2 <- data_for_logistic_reg2[!test_train_split_log_2, ]
```

Performing logistic regression

```r
logistic_reg_model_2 <- glm(is_popular ~ ., data = log_training_data_2, family = binomial)
summary(logistic_reg_model_2)
```

```
##
## Call:
## glm(formula = is_popular ~ ., family = binomial, data = log_training_data_2)
##
## Coefficients:
##                              Estimate Std. Error z value Pr(>|z|)    
## (Intercept)                 3.864e-02  1.093e-01   0.354  0.72361    
## self_reference_avg_sharess  8.880e-06  1.246e-06   7.127 1.03e-12 ***
## n_unique_tokens            -8.095e-01  3.463e-01  -2.337  0.01942 *  
## n_tokens_content            1.895e-04  3.744e-05   5.062 4.14e-07 ***
## kw_max_avg                 -9.780e-05  4.960e-06 -19.717  < 2e-16 ***
## n_tokens_title             -3.282e-02  6.117e-03  -5.366 8.07e-08 ***
## n_non_stop_unique_tokens    2.725e-01  2.912e-01   0.936  0.34932    
## kw_avg_min                  1.028e-04  3.821e-05   2.691  0.00713 ** 
## kw_avg_max                 -1.835e-06  1.219e-07 -15.052  < 2e-16 ***
## kw_avg_avg                  7.103e-04  2.317e-05  30.654  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 37120  on 29858  degrees of freedom
## Residual deviance: 35638  on 29849  degrees of freedom
## AIC: 35658
##
## Number of Fisher Scoring iterations: 5
```

Assessing the second model's performance

```r
predicted_prob_2 <- predict(logistic_reg_model_2, newdata = log_training_data_2, type = "response")
accuracy_2 <- mean(predicted_prob_2 >= 0.5)
cat("The accuracy rate of this model is", accuracy_2*100, "%")
```

```
## The accuracy rate of this model is 97.60541 %
```

Developing a random forest model

```r
# Using log_training_data as it has 46 variables independent variables and "is_popular" as the binary target vari
able
library(randomForest)
rf_model <- randomForest(is_popular ~ ., data = log_training_data, ntree = 500) # https://cran.r-project.org/web/
packages/randomForest/randomForest.pdf was used to understand Random Forest and implement the code
rf_model
```

```
##
## Call:
##  randomForest(formula = is_popular ~ ., data = log_training_data,      ntree = 500) 
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##         OOB estimate of  error rate: 28.23%
## Confusion matrix:
##      0     1 class.error
## 0 2648  6743   0.7180279
## 1 1667 18731   0.0817237
```

Developing a second random forest model with only the variables suggested by XGBoost

```r
# Using log_training_data_2 as it has the 9 independent variables suggested by XGBoost, and is_popular as the bin
ary target variable
rf_model_2 <- randomForest(is_popular ~ ., data = log_training_data_2, ntree = 500) # https://cran.r-project.org/
web/packages/randomForest/randomForest.pdf was used to understand Random Forest and implement the code
rf_model_2
```

```
##
## Call:
##  randomForest(formula = is_popular ~ ., data = log_training_data_2,      ntree = 500) 
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 30.26%
## Confusion matrix:
##      0     1 class.error
## 0 2220  7130  0.76256684
## 1 1905 18604  0.09288605
```

Evaluating the logistic regression classification model with 46 variables

```r
log_prob <- predict(logistic_reg_model, newdata = log_testing_data, type="response")
log_pred <- ifelse(log_prob>0.5, "1", "0")
log_pred<-as.factor(log_pred)
confusionMatrix(log_pred, log_testing_data$is_popular) # https://stackoverflow.com/questions/46028360/confusionma
trix-for-logistic-regression-in-r was referenced to make the confusion matrix
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0  810  673
##          1 2223 6149
##
##                Accuracy : 0.7061          
##                  95% CI : (0.697, 0.7151) 
##     No Information Rate : 0.6922          
##     P-Value [Acc > NIR] : 0.001393        
##
##                   Kappa : 0.1963          
##
##  Mcnemar's Test P-Value : < 2.2e-16       
##
##             Sensitivity : 0.26706         
##             Specificity : 0.90135         
##          Pos Pred Value : 0.54619         
##          Neg Pred Value : 0.73447         
##              Prevalence : 0.30776         
##          Detection Rate : 0.08219         
##    Detection Prevalence : 0.15048         
##       Balanced Accuracy : 0.58421         
##
##        'Positive' Class : 0               
##
```

Evaluating the logistic regression classification model with values from XGBoost

```r
log_prob_2 <- predict(logistic_reg_model_2, newdata = log_testing_data_2, type="response")
log_pred_2 <- ifelse(log_prob_2>0.5, "1", "0")
log_pred_2<-as.factor(log_pred_2)
confusionMatrix(log_pred_2, log_testing_data_2$is_popular) # https://stackoverflow.com/questions/46028360/confusi
onmatrix-for-logistic-regression-in-r was referenced to make the confusion matrix
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0  118  115
##          1 2956 6596
##
##                Accuracy : 0.6862          
##                  95% CI : (0.6769, 0.6953)
##     No Information Rate : 0.6858          
##     P-Value [Acc > NIR] : 0.4788          
##
##                   Kappa : 0.0284          
##
##  Mcnemar's Test P-Value : <2e-16          
##
##             Sensitivity : 0.03839         
##             Specificity : 0.98286         
##          Pos Pred Value : 0.50644         
##          Neg Pred Value : 0.69054         
##              Prevalence : 0.31415         
##          Detection Rate : 0.01206         
##    Detection Prevalence : 0.02381         
##       Balanced Accuracy : 0.51063         
##
##        'Positive' Class : 0               
##
```

### Evaluating the random forest model with 46 variables

```r
rf_model
```

```
##
## Call:
##  randomForest(formula = is_popular ~ ., data = log_training_data,      ntree = 500) 
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##         OOB estimate of  error rate: 28.23%
## Confusion matrix:
##      0     1 class.error
## 0 2648  6743   0.7180279
## 1 1667 18731   0.0817237
```

```r
# sensitivity: TP/(TP+FN)
# specificity: TN/(TN+FP)
library(pROC)
rf_predictions<- predict(rf_model, newdata = log_testing_data, type = "response")
rf_predictions<- as.numeric(rf_predictions)
roc_obj <- roc(log_testing_data$is_popular, rf_predictions) # https://cran.r-project.org/web/packages/pROC/pROC.p
df was referenced for this chunk of code
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
rf_sens<- coords(roc_obj, "best", ret = "sensitivity")
rf_spec<- coords(roc_obj, "best", ret = "specificity")
rf_sens
```

```
##   sensitivity
## 1   0.9204046
```

```r
rf_spec
```

```
##   specificity
## 1   0.2614573
```

### Evaluating the random forest model with 9 independent variables suggested by XGBoost

```r
# Evaluating the random forest model with features suggested by XGBoost
rf_model_2
```

```
##
## Call:
##  randomForest(formula = is_popular ~ ., data = log_training_data_2,      ntree = 500) 
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 30.26%
## Confusion matrix:
##      0     1 class.error
## 0 2220  7130  0.76256684
## 1 1905 18604  0.09288605
```

```r
rf_predictions_2<- predict(rf_model_2, newdata = log_testing_data_2, type = "response")
rf_predictions_2<- as.numeric(rf_predictions_2)
roc_obj_2 <- roc(log_testing_data_2$is_popular, rf_predictions_2) # https://cran.r-project.org/web/packages/pROC/
pROC.pdf was referenced for this chunk of code
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
rf_sens_2<- coords(roc_obj_2, "best", ret = "sensitivity")
rf_spec_2<- coords(roc_obj_2, "best", ret = "specificity")
rf_sens_2
```

```
##   sensitivity
## 1   0.9044852
```

```r
rf_spec_2
```

```
##   specificity
## 1   0.2433312
```

Benchmarking and comparing models

Developing a Naive Bayes model for benchmarking

```r
nb_model_1 <- naiveBayes(is_popular ~ ., data = log_training_data)
nb_model_2 <- naiveBayes(is_popular ~ ., data = log_training_data_2)

nb_predictions_1 <- predict(nb_model_1, newdata = log_testing_data)
nb_predictions_2 <- predict(nb_model_2, newdata = log_testing_data_2)

confusionMatrix(nb_predictions_1, log_testing_data$is_popular) # confusion matrix for Naive Bayes model with 47 v
ariables
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 2908 6397
##          1  125  425
##
##                Accuracy : 0.3382          
##                  95% CI : (0.3289, 0.3476)
##     No Information Rate : 0.6922          
##     P-Value [Acc > NIR] : 1               
##
##                   Kappa : 0.0134          
##
##  Mcnemar's Test P-Value : <2e-16          
##
##             Sensitivity : 0.9588          
##             Specificity : 0.0623          
##          Pos Pred Value : 0.3125          
##          Neg Pred Value : 0.7727          
##              Prevalence : 0.3078          
##          Detection Rate : 0.2951          
##    Detection Prevalence : 0.9442          
##       Balanced Accuracy : 0.5105          
##
##        'Positive' Class : 0               
##
```

```r
confusionMatrix(nb_predictions_2, log_testing_data_2$is_popular) # confusion matrix for Naive Bayes model with 9
variables suggested by XGBoost
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1977 3284
##          1 1097 3427
##
##                Accuracy : 0.5523          
##                  95% CI : (0.5424, 0.5622)
##     No Information Rate : 0.6858          
##     P-Value [Acc > NIR] : 1               
##
##                   Kappa : 0.1289          
##
##  Mcnemar's Test P-Value : <2e-16          
##
##             Sensitivity : 0.6431          
##             Specificity : 0.5107          
##          Pos Pred Value : 0.3758          
##          Neg Pred Value : 0.7575          
##              Prevalence : 0.3142          
##          Detection Rate : 0.2020          
##    Detection Prevalence : 0.5377          
##       Balanced Accuracy : 0.5769          
##
##        'Positive' Class : 0               
##
```

Comparing Naive Bayes model 1 with classification models with 46 variables

```r
model_comparison_1<- data.frame(
  Model = c("Naive Bayes", "Logistic Reg", "Random Forest"),
  Accuracy = c(0.3381, 0.7062, 0.7182),
  Sensitivity = c(0.9581, 0.2627, 0.9214),
  Specificity = c(0.0624, 0.9034, 0.2627    ))
model_comparison_1
```

```
##           Model Accuracy Sensitivity Specificity
## 1   Naive Bayes   0.3381      0.9581      0.0624
## 2  Logistic Reg   0.7062      0.2627      0.9034
## 3 Random Forest   0.7182      0.9214      0.2627
```

Comparing Naive Bayes model 2 with classification models with 9 variables

```r
model_comparison_2<- data.frame(
  Model = c("Naive Bayes", "Logistic Reg", "Random Forest"),
  Accuracy = c(0.5523, 0.6862, 0.6972),
  Sensitivity = c(0.6431, 0.0383, 0.9049),
  Specificity = c(0.5107, 0.98286, 0.2465   ))
model_comparison_2
```

```
##           Model Accuracy Sensitivity Specificity
## 1   Naive Bayes   0.5523      0.6431     0.51070
## 2  Logistic Reg   0.6862      0.0383     0.98286
## 3 Random Forest   0.6972      0.9049     0.24650
```

Evaluation and Conclusion

My primary objective with these two models was to obtain a good sensitivity value. Sensitivity is a model's ability to correctly predict a 'success', or popular article in this case. In this context, maximizing sensitivity is important as we want to ensure that popular articles can be accurately identified. The other 2 metrics assessed were specificity and accuracy. Specificity is the model's ability to correctly identify a failure, and accuracy is the fraction of the model's correct predictions.

In both comparisons (using 46 variables, and 9 variables), the Random Forest and the Logistic Regression models outperform Naive Bayes in terms of accuracy. I will focus on the model with 9 variables, as this simpler model will be less prone to overfitting.

Considering the models developed using 9 features, Random Forest outperforms the other 2 models in terms of accuracy, and more importantly, sensitivity. In context, a sensitivity of 0.9049 means that an estimated 90.49% of the articles that are genuinely popular are correctly classified as popular by the model. Thus, the model likely won't miss out on many popular articles, and Mashable will be able to benefit from the higher amount of revenue generated for each additional share.

For this reason, I recommend the Random Forest model to be used to select articles.